

Fiori Elements 開發課程 13 (延伸篇) : Value Help 真實驗證

環境：BTP ABAP Environment Trial (沿用 fe08 已經產生的 `fe08taskprojection` 專案，不新增任何 ABAP 物件)

Lecture

這一課要補的洞

課程總覽時發現一個具體的缺口：rc08 (RAP Cloud 課程) 已經完整建好一個 Value Help (`ZI_RC08_STATUS_VH` Custom Entity + `ZCL_RC08_STATUS_VH` Query Provider)，掛在 `ZI_RC01_TASK` 的 `status` 欄位上——但驗證只做到 Eclipse 內建 Swagger / Service Binding Preview (rc08 講義：「ZRC07_SB Preview：status 欄位篩選有 Value Help 下拉選單」)。

fe08 這一課用的正是同一個 `ZI_RC01_TASK` (透過 `ZC_RC01_TASK` Projection)，也把 `status` 欄位曝露出來，但講義從頭到尾沒有提到去點那個 F4 下拉選單、截圖確認——這門課從第一課就強調「Eclipse Preview 只是臨時預覽，要在真正的 VS Code App 裡驗證才算數」，這裡是唯一一個說到沒做到的地方。這一課要把這個洞補上。

好消息：後端完全不用碰，物件都已經在

查證 `zi_rc01_task.ddlx.abap` 的完整內容，`status` 欄位掛了兩個關鍵 annotation：

```
@UI.selectionField: [ { position: 20 } ]
@UI.lineItem: [
  { position: 30 },
  { type: #FOR_ACTION, dataAction: 'markDone', label: 'Mark Done' }
]
@UI.identification: [
  { position: 30 },
  { type: #FOR_ACTION, dataAction: 'markDone', label: 'Mark Done' }
]
@Consumption.valueHelpDefinition: [ { entity: { name: 'ZI_RC08_STATUS_VH', element: 'status' } } ]
status;
```

- `@UI.selectionField`——代表 `status` 會直接出現在 List Report 篩選列，不用先去「Adapt Filters」才找得到
- `@Consumption.valueHelpDefinition`——指向 `ZI_RC08_STATUS_VH`，這個 Value Help 回傳 O/Open、D/Done 兩筆資料 (rc08 已用 Swagger 驗證過協定層正確)

fe08 建立 `ZC_RC01_TASK` 的 Metadata Extension 時，是把 `ZI_RC01_TASK` 這份 DDLX 內容整個複製過去 (fe08 講義原文：「欄位定義一字不改」)，所以 `ZC_RC01_TASK` 身上也有同一份 `status` annotation——

fe08taskprojection 這個已經存在的 VS Code 專案，理論上打開就看得這個 Value Help，不需要任何新建立或修改的物件。

⚠ 容易誤解的一點：這不是「Projection 自動繼承 Interface 的 Annotation」

值得先講清楚一個技術細節，避免以後對兩層架構的機制產生錯誤預期：**CDS Metadata Extension (DDLX)** 是綁定特定 **CDS Entity** 名稱的獨立物件，**ZC_RC01_TASK as projection on ZI_RC01_TASK** 這個 **projection on** 關係，不會讓 **ZC_RC01_TASK** 自動拿到 **ZI_RC01_TASK** 身上掛的 **DDLX** 內容。fe08 能讓 **ZC_RC01_TASK** 也有 UI Annotation，靠的是人工複製一份新的 **DDLX**，目標換成 **ZC_RC01_TASK**，不是框架自動幫你做的——這代表如果之後 **ZI_RC01_TASK** 的 **DDLX** 改了（例如加一個新欄位的 annotation），**ZC_RC01_TASK** 的 **DDLX** 不會自動同步，要手動改兩份。這跟 CDS View 本體的欄位（**select from**）透過 **as projection on** 真的會自動繼承，是兩種不同的機制，容易搞混。

動手操作：在真正的 VS Code App 裡打開 F4

沿用 **fe08** 已經產生的專案，不用重新跑 **Generator**：

1. 開終端機，**cd** 進 **src/ABAP_Training_Fiori_Elements/fe08taskprojection/**
2. 執行 **npm start**（如果 fe08 那次的伺服器還在背景跑，直接切到那個分頁即可，不用重開）
3. 瀏覽器開啟 List Report（**TaskList**），畫面上方篩選列應該直接看得到 **Status** 這個篩選欄位（因為 **@UI.selectionField**，不用先點「Adapt Filters」）
4. 點 **Status** 欄位右側的下拉 / 搜尋圖示，觸發 Value Help
5. 預期畫面：跳出一個對話框（「Define Conditions」或「Search and Select」分頁），列出兩筆資料：**O / Open**、**D / Done**——這兩筆資料完全是 **ZCL_RC08_STATUS_VH** 這個 Query Provider Class 在程式碼裡動態組出來的，不是查資料庫表
6. 選 **Done**，確認套用，List Report 應該只剩下 **status = 'D'** 的 Task
7. 額外實驗：清掉篩選，這次不用 Value Help 下拉，直接在 Status 欄位手動打字（例如打 **X**，一個 Value Help 清單裡沒有的值），觀察畫面允不允許——這能驗證一件事：
@Consumption.valueHelpDefinition 預設只是「提供建議清單」，不是「強制只能選清單內的值」（要做到強制檢核，需要額外的 Validation，這正好也是 fe14 要教的機制）

如果 Status 欄位在篩選列沒有出現

@UI.selectionField 理論上會讓欄位直接顯示，但如果實際畫面沒看到，先點篩選列的 **Adapt Filters**（漏斗圖示），確認 **Status** 有沒有被勾選——這不代表 annotation 沒生效，只是畫面預設顯示的篩選欄位數量 / 版面配置可能受畫面寬度或其他設定影響，手動勾選一樣能達到相同的驗證目的。

學習目標

- 能講出 **@UI.selectionField** 跟 **@Consumption.valueHelpDefinition** 各自的作用：前者決定欄位要不要直接出現在篩選列，後者決定這個欄位有沒有 F4 下拉建議
- 知道 CDS Metadata Extension (DDLX) 不會透過 **as projection on** 自動從 Interface 傳給 Projection——兩層架構下，UI Annotation 要嘛複製一份、要嘛只掛在其中一層曝露的那個
- 知道 **@Consumption.valueHelpDefinition** 預設只是「建議清單」，不是「強制檢核」——這兩者是不同機制，別搞混
- 能講出這一課驗證的意義：Eclipse Preview / Swagger 只能驗證協定層跟資料正確性，UI 元件（Value Help 對話框）實際怎麼呈現、好不好用，只有在真正的 App 裡才看得出來——這正是 fe01 開課動機那段話的具體體現

物件清單

這一課沒有新增或修改任何物件——完全沿用既有的：

物件	來源	角色
<code>ZI_RC08_STATUS_VH</code>	rc08 (RAP Cloud 課程)	Value Help Custom Entity
<code>ZCL_RC08_STATUS_VH</code>	rc08	Value Help Query Provider
<code>ZI_RC01_TASK</code> 的 <code>@Consumption.valueHelpDefinition</code>	rc08	掛在 Interface View 上的 annotation
<code>ZC_RC01_TASK</code> 的同份 annotation (複製)	fe08	掛在 Projection View 上的 annotation
<code>fe08taskprojection</code>	fe08	這一課直接拿來用的既有 VS Code 專案

動手練習

- 打開瀏覽器開發者工具的 Network 分頁，點 Value Help 觸發下拉選單時，找出實際打出去的 OData 請求網址（提示：路徑會包含 `/StatusVH`）——確認這個請求走的是不是 `$batch`（呼應 rc08 講義提到的「Fiori Elements 元件透過 `$batch` 呼叫」這句話）
- 想一想：如果之後要讓 `status` 變成「只能選清單內的值，使用者不能手動亂打」，你會怎麼設計？（這是 fe14 要教的 Validation 機制的預告，先自己想一下要怎麼擋）
- 對照 fe03 學過的「本機 `annotation.xml` 疊加」機制：如果你想在**不改 ABAP**的前提下，讓 `fe08taskprojection` 的 Status 篩選欄位額外多顯示一個 `status_text` 的說明文字，有沒有辦法用本機疊加做到？（提示：`@Consumption.valueHelpDefinition` 也可以在 `entity` 底下加 `element` 對應多個欄位）

驗證方式

✅ 已由使用者在 **VS Code** 實機驗證通過（2026-08-24）：

- `fe08taskprojection` 的 List Report 篩選列有 **Status** 欄位
- 點下拉圖示跳出 Value Help 對話框，看到 `Open` / `Done` 兩筆
- 選一筆套用後，List Report 資料確實被篩選

rc08 只驗證到 Eclipse Preview 的這個 Value Help，到這一課才第一次在真正的 VS Code App 裡確認可用——呼應這門課從 fe01 就強調的「Eclipse Preview 只是臨時預覽，要在真正的 App 裡驗證才算數」。

思考題

- `ZI_RC08_STATUS_VH` 是 `define custom entity`，不對應任何資料庫表。如果之後 Task 的合法狀態值變多（例如加一個 `In Progress`），要改的是哪個物件？需不需要動到 `ZI_RC01_TASK` 本身？
- 這一課示範的 Value Help 資料是寫死在 ABAP 程式碼裡（`VALUE #( ( status = 'O' status_text = 'Open' ) ( status = 'D' status_text = 'Done' ) )`）。如果改成真的查一張 Customizing 表，

ZCL_RC08_STATUS_VH 的 `if_rap_query_provider~select` 方法大概要怎麼改？（提示：rc08 講義提過的「完整性檢查」機制，換成真的查表後還要不要繼續呼叫那些 `getter`？）

答案

這一課不產生新的答案物件——驗證結果就是上面「驗證方式」三項有沒有在你的畫面上成立。若跟預期不符，回頭比對 `zi_rc01_task.ddlx.abap` (`status` 欄位的 `annotation`) 與 `fe08taskprojection/webapp/manifest.json` (`Task Entity` 的 `contextPath` 設定) 逐項排查。